Library Management System

A Role-Based Database Application using Python & MySQL

PROJECT REPORT

Submitted by:

Geetesh Parashar

Registration No: 25MIP10146

Course Name:

Python Essentials

Date of Submission:
31 March 2026

Department of Computer Science & Engineering

Contents

1 Introduction **2**

2 Problem Statement **2**

3 Functional Requirements **2**
3.1 Administrator Module....2
3.2 Librarian Module....2
3.3 Member Module....2

4 Non-functional Requirements **3**

5 System Architecture **3**

6 Design Diagrams **3**
6.1 Use Case Diagram....3
6.2 Sequence Diagram (Checkout Process)....3
6.3 ER Diagram (Entity-Relationship)....3

7 Design Decisions & Rationale **4**

8 Implementation Details **4**

9 Screenshots / Results **4**

10 Testing Approach **5**
10.1 Unit Testing....5
10.2 Integration Testing....5

11 Challenges Faced **5**

12 Learnings & Key Takeaways **5**

13 Future Enhancements **5**

14 References **6**

1 Introduction

The **Library Management System (LMS)** is a comprehensive software solution designed to automate the administrative and operational tasks of a modern library. Built using **Python** for the application logic and **MySQL** for robust data persistence, this system facilitates the management of books, user accounts, and circulation (loans/returns). It features a secure, role-based login system that provides distinct functionalities for Administrators, Librarians, and General Members.

2 Problem Statement

Traditional manual library management involves tedious paperwork, making it difficult to track book inventory, user borrowing history, and overdue returns efficiently. This often leads to data redundancy, loss of books, and administrative errors. The goal of this project is to develop a digitized system that centralizes all library data, automates the calculation of due dates, ensures data security through authentication, and provides real-time reporting on library status.

3 Functional Requirements

The system implements a Role-Based Access Control (RBAC) model with specific requirements for each user type:

3.1 Administrator Module

- **User Management:** Add new users, update user details, view all users, and suspend accounts.
- **Book Management:** Add new book titles (including total copies), update book details, and remove books from the catalog.
- **Reporting:** Generate aggregated reports on book genres, user roles distribution, and loan statuses.

3.2 Librarian Module

- **Checkout Process:** Issue books to members, automatically updating inventory and setting due dates.
- **Return Process:** Accept returned books, calculate overdue status, and update inventory availability.
- **Monitoring:** View currently active loans and identify overdue books.

3.3 Member Module

- **Catalog Search:** Search for books by title, author, or genre.
- **My Account:** View personal borrowing history, check due dates, and calculate remaining days for returns.

4 Non-functional Requirements

- **Security:** Passwords are never stored in plain text; they are hashed using **SHA-256** before storage.
- **Data Integrity:** MySQL transactions ensure that book inventory counts and loan records remain consistent (e.g., inventory decreases only when a loan is successfully created).
- **Usability:** A clean, menu-driven Command Line Interface (CLI) utilizing `tabulate` for readable data presentation.
- **Scalability:** The database schema is normalized to support a growing number of books and users without performance degradation.

5 System Architecture

The project follows a classic **Two-Tier Architecture**:

1. **Application Tier (Python):** Contains the business logic, input validation, menu navigation, and database interface class `LibrarySystem`.
2. **Data Tier (MySQL Server):** Stores persistent data in three relational tables: `users`, `books`, and `loans`.

6 Design Diagrams

6.1 Use Case Diagram

The diagram visualizes the three actors (Admin, Librarian, Member) and their respective authorized actions.

6.2 Sequence Diagram (Checkout Process)

- **Librarian** inputs Book ID and User ID.
- **System** verifies Book availability (> 0) and User status (Active).
- **System** checks User's current loan count (< 5).
- **System** executes SQL Transaction: Insert into `loans` table AND Decrement `available_copies` in `books` table.
- **Database** confirms transaction.
- **System** displays success message with Due Date.

6.3 ER Diagram (Entity-Relationship)

The database schema consists of three normalized tables.

Table 1: Database Schema Relationships

Table	Primary Key	Foreign Keys / Relationships
users	user_id	None. Stores credentials and roles.
books	book_id	None. Stores inventory data.
loans	loan_id	Links to books(book_id) and users(user_id).

7 Design Decisions & Rationale

- **Object-Oriented Design:** The system is encapsulated in a single class LibrarySystem. This maintains state (like self.current_user) and keeps database methods organized.
- **Password Hashing:** Implemented hashlib.sha256 to ensure that even if the database is compromised, user passwords remain secure.
- **Constraint Enforcement:** Business rules (e.g., "Maximum 5 books per user") are enforced in the Python logic before database modification to prevent policy violations.
- **Input Masking:** Used getpass for password entry to prevent credentials from being visible on the screen during typing.

8 Implementation Details

- **Database Initialization:** The initialize_database() method checks for the existence of tables and creates them if missing. It also auto-generates a default "admin" account if the user table is empty.
- **Transaction Management:** Operations involving multiple updates (like checking out a book) utilize autocommit=False logic (implicit in the connector unless set otherwise) and explicit self.connection.commit() calls to ensure atomicity.
- **Date Handling:** The datetime and timedelta modules are used to accurately calculate due dates (14 days from checkout) and check for overdue items.
- **Dynamic Menus:** The run() loop dynamically calls admin_menu, librarian_menu, or member_menu based on the role attribute of the logged-in user.

9 Screenshots / Results

(In the final report, place actual screenshots of the console here)

- Screenshot 1: Successful Admin Login and Dashboard display.
- Screenshot 2: The "View Reports" screen showing tabulated data of books and users.
- Screenshot 3: A Librarian successfully checking out a book, showing the calculated due date.
- Screenshot 4: A Member searching for a book and seeing the search results table.

10 Testing Approach

10.1 Unit Testing

- Tested the hash_password function to ensure consistent hash generation.
- Verified date calculations for due dates (Check date + 14 days).

10.2 Integration Testing

- **Workflow Test:** Registered a new Member (as Admin) → Logged in as Librarian → Checked out book to new Member → Logged in as Member → Verified book appears in ”My Books”.
- **Constraint Test:** Attempted to borrow a 6th book for a user with 5 active loans; verified the system rejected the request.
- **Inventory Test:** Verified that checking out a book decreases available_copies by exactly 1.

11 Challenges Faced

- **Handling Dates in SQL:** Mapping Python datetime objects to MySQL DATE columns required careful formatting and understanding of the connector’s data type conversion.
- **State Management:** Ensuring that the self.current_user is correctly cleared upon logout to prevent unauthorized access by subsequent users of the terminal.
- **User Experience:** formatting the console output to be readable was difficult initially; integrating the tabulate library solved this by producing clean, grid-style ASCII tables.

12 Learnings & Key Takeaways

- Gained deep insight into **SQL Join operations** required to link the ‘loans‘, ‘books‘, and ‘users‘ tables for generating meaningful reports.
- Learned the importance of **Role-Based Access Control (RBAC)** in designing secure enterprise applications.
- Understood **ACID properties** in databases by implementing transactional logic for book circulation.
- Improved proficiency in **Python Object-Oriented Programming (OOP)** by structuring the application as a cohesive class.

13 Future Enhancements

- **Email Notifications:** Integrate an SMTP client to send automated email reminders to users one day before their books are due.
- **Fine Management:** Implement a system to calculate and track monetary fines for overdue books (e.g., $1 per day overdue).

- **Barcode Integration:** Add support for a barcode scanner input to speed up the check-out/return process for physical books.
- **Web Interface:** Port the application to Flask or Django to provide a browser-based interface accessible over the network.

14 References

- Python 3 Documentation (`datetime`, `hashlib`).
- MySQL Connector/Python Developer Guide.
- ”Database System Concepts” by Silberschatz, Korth, and Sudarshan.
- PyPI Documentation for `tabulate` library.